EPSR-test-ex60 clusters: three distributions, accumulated over 2 runs

Program(s): clusters | chained passes: 2 | verification: ALL IDENTICAL (all 2 passes)

What this case tests

Exercises `clusters`, the cluster analysis of EPSR [1]: two molecules are bonded if an atom1-atom2 pair lies within [rmin, rmax], bonded molecules form clusters, and the program writes the cluster-size histogram, the fraction of molecules in percolating clusters (spanning the box in at least one dimension) and the fraction in clusters of two or more molecules. This case checks the accumulation over runs.

Code exercised

Fortran (original, fortran_src/):

- `clusters.f`: `run_clusters`, `find_clusters` (bond list, cluster labelling, percolation test)
- `read_and_save_files.f`: `read_input_file`, `outgofz`, `save_files_others`; `auxiliary_routines.f`: `r_int`
- `setup_symbols_generic.f`: `setup_symbols_clusters`

Rust (port, rust/epsr-core/src/):

- `tools/clusters.rs`: `run`, `find_clusters`, `outgofz`, `open_input_with`
- `symbols.rs`: `setup_input_file`, `read_input_file`, `save_inp_file`; `output.rs`: `r_int`; `symtab_tools.rs`: `CLUSTERS`

Physical system

Analysed configuration: `200DMSO400Water.ato`, an unchanged copy of the EPSR-test box of 200 dimethyl sulphoxide (DMSO; atom types S, O, C, H) and 400 water molecules (O1, H1): 3200 atoms, DMSO mole fraction 1/3, 300 K, cubic box of side 33.1213 Å (0.0881 atoms/Å^3), the EPSR model of the deuterated DMSO-water neutron data (ISIS SANDALS runs SLS03863, SLS03871, SLS03850), cf. [2]. Distributions: (1) water O1···H1 hydrogen bonds 1.4-2.4 Å, (2) DMSO O···water H1 1.4-2.5 Å, so molecules of both types take part, (3) S···O1 contacts 3.0-4.0 Å. In the reference, 88 % of the molecules of distribution 1 are in percolating clusters, and the fractions in clusters of 2 or more are 0.907, 0.650 and 0.775.

Inputs

`hb.CLUSTERS.dat`: `nsize 60`, `nsumt 0`, three distributions as above. `200DMSO400Water.ato` copied unchanged from EPSR-test; the keyword file was written by hand (the Fortran program creates no template when the named file does not exist; keywords and defaults from `setup_symbols_generic.f`).

How the accuracy was verified

`commands.txt` (1 program call, repeated 2 times, each pass reading the files written by the previous one) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in all 2 passes.
`repeat 2`: run 1 starts an accumulation, run 2 reads `hb.CLUSTERS.n01` back and averages (`outgofz` accumulation branch, non-zero error column). Compared after each pass: `hb.CLUSTERS.n01` (row 1 = percolating fraction, rows 2.. = normalised cluster-size histogram), the rewritten `hb.CLUSTERS.dat` (`nsumt` 1, then 2) and `fort.14`. The configuration is the same in every pass, so the chained passes test the read-back of the accumulated file and the running-average arithmetic.

Notes

`fort.14`: the Fortran writes the fraction lines to unit 14 after `save_files_others` has closed it, so gfortran writes `fort.14` in the working directory; the Rust port writes it to the case folder.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] A. K. Soper and A. Luzar, "A neutron diffraction study of dimethyl sulphoxide-water mixtures", J. Chem. Phys. 97, 1320-1331 (1992).
[3] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.